AuraRoute AI

Security Self-Assessment Report: Security & Compliance Infrastructure, Permission Tiers and Known-Risk List

2026 Shenzhen FinTechathon, International Track. Topic C: Cross-border & Student Finance Assistant

Submission Item 05 | Team: AuraRoute AI

5.1 Scope and Method

This report assesses the AuraRoute AI prototype described in the Technical Documentation (Item 01). AuraRoute AI is an explainable, read-only, non-custodial, compliance-first advisory agent: as a financial participant it completes a real task (planning a cross-border tuition payment) without ever holding or moving money. The assessment covers the code in the repository (auraroute_core.py and run_demo.py), the sample data, and the data flows between the stages of the pipeline. It is a self-review by the team: we reviewed each control against the code, exercised it with unit tests, and recorded an end-to-end run in the execution evidence log (Item 06). It is not an independent audit or a penetration test.

The prototype is advisory and read-only. It runs on recorded fixtures by default and can use Fixer.io, Plaid Sandbox and an optional model provider when credentials are supplied. It handles no real funds and, in demonstration, no real personal data: the sample invoice is synthetic.

5.2 Permission Tiers

Capabilities are divided into tiers. Each tier lists what the system may do, what it is not allowed to do, and how that is enforced in the implementation.

Tier	Permitted	Not permitted	Enforcement
Tier 0. Money movement	None.	Holding, moving, routing or splitting funds. Initiating payments. Writing to any bank or payment endpoint.	No such code exists. The only network code is in two wrapper functions (FX rate lookup and balance read). The one POST request is Plaid’s balance read.
Tier 1. External read-only	Exchange-rate lookup (Fixer.io). Balance read (Plaid Sandbox) using read-only API sandbox access tokens.	Any other endpoint, any write operation, any call made directly by the model.	Only fetch_fx_rate and fetch_sandbox_balances touch the network. Credentials come from environment variables. If a live call fails, a recorded fixture is used and an audit event is written.
Tier 2. Local processing	Redaction, parsing, rule retrieval, advisory checks and audit logging on redacted text and typed fields.	Processing unredacted identifiers. Persisting raw invoice text.	Redaction runs first in parse_and_sanitize_invoice. Only the redacted text and typed fields continue downstream. The audit log stores hashes of event details, not the details.
Tier 3. Human approval	The student confirms or corrects parsed values.	Any downstream use of unconfirmed values.	human_in_the_loop_gate sets the verified flag only after validation. execute_compliance_routing_check refuses unverified payloads and records the attempt.
AI model (optional, any use)	Fill extraction fields the regular expressions missed, from redacted text.	Tool access, network access, overwriting parsed values, marking a payload verified, changing a compliance result.	llm_fallback_extract fills missing fields only. validate_llm_extraction discards output with unknown keys, invalid values or non-JSON content.

5.3 Controls and Evidence

Control	How it is verified	Evidence
Redaction of eight labeled identifier types: name, passport number, phone, email, government/national ID, address, student ID, date of birth	Unit tests check that multi-word names and every labeled identifier are removed, and that the institution line is left intact.	tests/test_core.py; tests/test_extended_redaction.py; redacted text in execution_evidence_logs.txt
Human gate before analysis	Tests check that the gate refuses missing fields, non-positive amounts and invalid dates, and that the orchestrator blocks unverified payloads and logs the attempt.	tests/test_core.py; blocked-payload line in the evidence log
No structuring; quota tracking	Tests confirm that an over-quota amount is flagged, produces advice to use the bank's documented tuition-payment route (admission letter and tuition certificate), and contains no split action.	tests/test_core.py; tests/test_extended_redaction.py; evidence log
Model output validation	Tests feed invalid, hostile and malformed model output and confirm it is rejected, and confirm that parsed values are never overwritten.	tests/test_core.py
Audit trail integrity	A test alters an earlier entry and confirms that chain verification fails.	tests/test_core.py; chain verification line in the evidence log
Failure handling	A test supplies an unusable exchange-rate key and confirms the fixture fallback and the fx_source label.	tests/test_core.py
Honest labeling of thresholds	The USD 2,500 living-expense figure is labeled an illustrative framework baseline in the code, deck and documentation, and the advisory note says real requirements vary.	auraroute_core.py comments and advisory note; Items 1 and 2

Unit tests run with: python -m unittest discover -s tests -v

5.4 Data Handling

- Uploaded invoices are processed locally. Identifying lines are removed before extraction, so downstream code and any model call see only redacted text.
- The audit log keeps timestamps, event names and hashes, not invoice contents or identities.
- If a model provider is configured, only redacted text is sent. Free-tier provider terms may allow submitted content to be used to improve products, so only synthetic or redacted text should be used with a free key.
- API keys and Plaid credentials are read from environment variables. The .env file is excluded from version control and only .env.example is committed.
- The dashboard HTML is generated with escaped values, so text taken from an invoice cannot inject markup.
- The prototype has no user accounts and no database. Nothing is stored beyond the local output files created by a run.

5.5 Known-Risk List

Residual risk is the team's judgment after the listed mitigation. The list is deliberately candid: we publish residual ratings, including the risks we have reduced but not eliminated.

ID	Risk	Mitigation in the prototype	Residual risk and status
R1	Parsing errors from unusual layouts or OCR mistakes	Missing fields stay empty instead of defaulting. The student must confirm every value at the gate. A confidence score is shown.	Low. The student can still confirm a wrong value without noticing.

ID	Risk	Mitigation in the prototype	Residual risk and status
R2	Identifiers in formats the redaction patterns do not cover	Patterns cover labeled lines for name, passport number, phone, email, government/national ID, address, student ID and date of birth.	Medium. Unlabeled or free-text identifiers can pass through. Planned: broader detection.
R3	Prompt injection through text in an uploaded invoice	The model receives redacted text only and has no tool or network access. Its output is validated and can fill only missing fields. Compliance results are computed by code, and the human gate sits before any analysis.	Medium. Risk is reduced, not eliminated. A model could still propose a wrong value that passes validation, which the student then has to catch at the gate.
R4	Incorrect or outdated regulatory information	Thresholds live in a rules table with a source note. Retrieved snippets carry source and section. Outputs are labeled advisory.	Medium. The sample corpus is small and static. Figures must be verified against current official text before real use.
R5	Exchange-rate or balance service outage	On live-call failure the system uses a recorded fixture, writes an audit event and labels the result with its rate source.	Low to medium. Fixture rates are not current, so the label matters. Planned: show rate freshness on the dashboard.
R6	User over-reliance on advisory output	Dashboard and notes state that the output is planning information, not legal or immigration advice, and point to the bank, institution or consulate.	Medium. Presentation cannot fully prevent over-reliance.
R7	Advice that encourages evasion of limits	The system never proposes splitting a payment. Over-quota cases are flagged and directed to the bank's documented tuition-payment route.	Low.
R8	Credential leakage	Environment variables only. .env is git-ignored and only .env.example is committed. No credentials appear in logs.	Low. Depends on the team's repository hygiene. Planned: automated secret scanning.
R9	Audit log tampering	Hash-chained entries make alteration of earlier entries detectable.	Medium. The log is tamper-evident, not tamper-proof, and the file can be deleted.
R10	Third-party dependency risk	The core uses the Python standard library. The optional requests library is the only dependency.	Low. Planned: dependency scanning.
R11	Prototype has no authentication or multi-user separation	Out of scope for the prototype. It runs locally for one user and stores nothing beyond local output files.	High if deployed as a service. Must be added before any production use.
R12	Third-party model provider handles submitted text	Only redacted text is sent. The adapter is off unless a provider and key are set. Output is validated before use. Adapter requests are covered by mocked tests that assert identifiers never appear in the request.	Medium. Provider data-use terms apply. Free tiers may use content to improve products, so use synthetic data only.
R13	Sandbox and fixture data may not reflect real accounts	Outputs name their data source. Live behavior depends on the bank and country supported by Plaid.	Medium. Coverage for a given student's bank is not guaranteed.
R14	The USD 2,500 balance baseline is far below real maintenance requirements and living costs, which vary by country, city and time	Labeled an illustrative framework baseline everywhere it appears. It is a configurable constant, and the advisory note tells the student to check maintenance-fund requirements with the institution or consulate.	Medium. Not an official figure. Destination-specific, current figures must replace it before real use.

5.6 Planned Improvements

- Add authentication, per-user data separation and encryption at rest before any deployment beyond a local prototype.
- Replace the paraphrased rule entries with verified excerpts that include article numbers, and add embedding-based retrieval with source citations.

- Broaden PII detection beyond labeled lines and add a review step that shows the redacted text before upload.
- Show exchange-rate freshness on the dashboard, refresh the rules table on a schedule, and replace the illustrative baseline with destination-specific figures.
- Add secret scanning and dependency scanning to the repository workflow.